

CIPHER 用户手册

EIC7x 系列 AI 数字 SoC

Engineering Draft / Rev0.7
2025-07-02

声明

版权所有©2024，北京奕斯伟计算技术有限公司及其关联公司（以下简称“ESWIN”）。保留所有权利。

ESWIN 在此软件中保留所有知识产权和专有权利。未经 ESWIN 授权，不得发布、复制、分发、转载、修改、改编、翻译或制作此软件的衍生作品，无论是全部还是部分。

对于本文档的任何修改，ESWIN 无需承担通知义务且不代表 ESWIN 做出任何承诺。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证与承诺，无论明示或默示。（本段落用于文档中）

“ESWIN” logo 和其他 ESWIN 标识，为北京奕斯伟计算技术有限公司及（或）其母公司所有的商标。

本文档提及的其他所有商标或商品名称，由其各自的所有权人所有。

产品安装中可能包含开源软件和/或自由软件的许可声明。

Change History

Version	Date	说明
V0.7	2025-07-02	增加生成 loadable 程序说明
V0.6	2024-04-08	部分英文改为中文
V0.5	2024-02-27	
V0.4	2023-12-4	Initial edition, an engineering draft.
V0.4	2024-09-11	修改副标题为"EIC7x 系列 AI 数字 SoC"
V0.4	2024-11-29	文档名称改为 "CIPHER 用户手册_CN_v0.4.docx" 正文中 WIN2030 字样改为 EIC7x

目录

CIPHER 用户手册	0
EIC7x 系列 AI 数字 SoC.....	0
目录.....	III
表目录.....	IV
图目录.....	V
1. 概述	1
1.1 使用流程.....	2
1.1.1 对称加解密.....	2
1.1.2 HASH 计算.....	3
1.1.3 HMAC 计算.....	3
1.1.4 产生随机数.....	4
1.1.5 RSA 加解密.....	4
1.1.6 NsignVerify 签名验证.....	5
1.1.7 Image 解密.....	6
1.1.8 ECDH Key 协商.....	7
1.1.9 更新 RAM 公钥.....	7
1.1.10 下载扩展服务程序.....	8
1.1.11 RSA 签名及验签操作步骤.....	9
2. API 参考	10
3. 数据类型	27
4. 错误码	33
附一、制作 LOADABLE 程序	33

表目录

表 1-1 不同算法支持的工作模式	1
表 4-1 错误码.....	33

图目录

图 1- 1 CIPHER 框架 2

图 1- 2 扩展服务-签名/验签 流程图..... 9

图 1- 3 nsign 签名后的数据运行演示 34

1. 概述

CIPHER 是奕斯伟 EIC7x 系列芯片提供的安全算法模块，其提供了包括 AES、DES/3DES 和 SM4 等对称加解密算法，RSA、ECSDA 不对称加解密算法，随机数生成，以及支持 HASH、HMAC、MAC 等摘要算法，主要用于对音视频码流进行加解密保护，用户合法性认证等场景。不同算法支持的工作模式如下表所示：

表 1-1 不同算法支持的工作模式

算法	支持的工作模式
AES	ECB/CBC/CTR/ CFG/OFB/CS1/CS2/CS3；注：通过加载 service code 的方式，可以支持 CCM/GCM 模式
DES/3DES	CBC/ECB
SM4	ECB/ CBC/ CFB/ OFB/ CTR/ CS1/CS2/CS3；注：通过加载 service code 的方式，可以支持 CCM/GCM 模式
RSA	1024/2048/4096
ECSDA	NIST_P256/ NIST_P384/ NIST_P521/ BRAINPOOL_P256R1/ BRAINPOOL_P384R1/ BRAINPOOL_P512R1
HASH	SHA1/SHA224/SHA256/SHA384/SHA512/SHA512_224/SHA512_256/MD5/SM3
HMAC	HMAC_SHA1/HMAC_SHA224/HMAC_SHA256/HMAC_SHA384/HMAC_SHA512/HMAC_SHA512_224/HMAC_SHA512_256/HMAC_MD5/HMAC_SM3
随机数	TRNG

CIPHER 模块提供了 NSIGN 验签功能。需要配合使用奕斯伟提供的 NSIGN 签名打包工具生成签名。

CIPHER 模块还提供了 NSIGN 包解密功能。该功能先验证签名，如果数据是密文，会同时解密生成明文。需要配合使用奕斯伟提供的 NSIGN 签名打包工具生成签名及对数据加密。

CIPHER 模块还支持加载和运行一段经过验签的 firmware。Firmware 需要使用 NSIGN 签名打包工具生成签名。

上述算法及功能都是以 service 服务的方式，向用户提供服务 and 接口。CIPHER 模块也支持加载和运行用户的 service code，从而提供用户自定义的 service 功能。该 service code 同样需要使用 NSIGN 签名打包工具生成签名。如图 1-1 所示：

Security CPU 启动时，会读取 OTP Security Enable 标记位。如果标记位是 security mode，SCPU 会执行 security boot 流程，即下一级 boot image(一般是 opensbi+uboot)做验签，下一级 boot image 必须是密文，验签和解密通过后会才会执行 boot image。同时，如果是 security mode，本文所描述的所有 service 服务都支持。如果标记位是 un-security mode，SCPU 不会对下一级 boot image 做验签，下一级 boot image 必须是明文。同时，如果是 un-security mode，以下四个 service 服务不支持：NsignVerify 签名验证、Image 解密、更新 RAM 公钥、下载扩展服务程序。

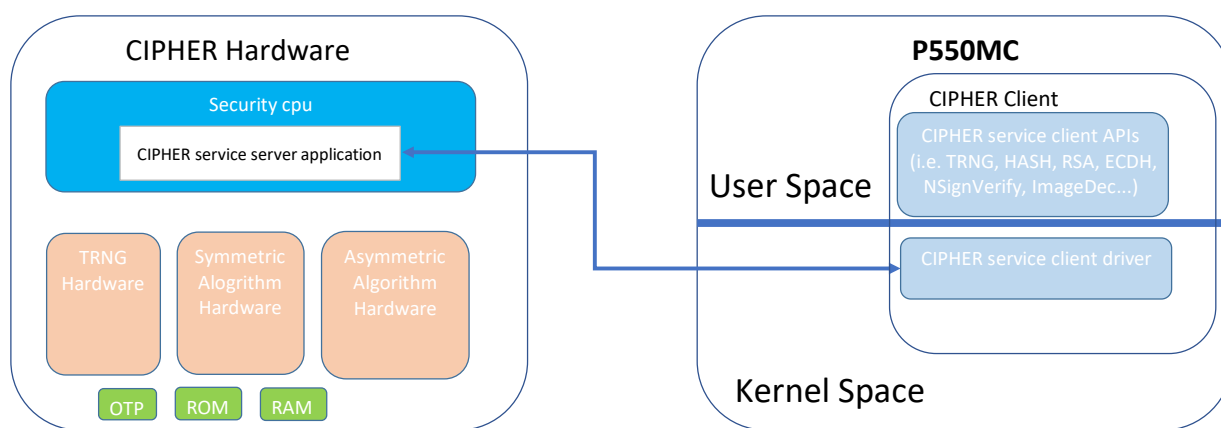


图 1- 1 CIPHER 框架

1.1 使用流程

1.1.1 对称加解密

工作流程

对数据进行对称的 DES/3DES/AES/SM4 加解密的过程如下：

步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。

步骤 2 配置 CIPHER 控制信息，包含密钥、初始向量、加密算法、工作模式等信息，并获取该控制信息的 CIPHER 句柄。调用接口 [ES_CIPHER_CreateHandle](#) 完成。

步骤 3 对数据进行加解密。

加密-- [ES_CIPHER_Encrypt](#) 当拆分成多个包加密时，多次调用该接口。

解密-- [ES_CIPHER_Decrypt](#) 当拆分成多个包解密时，多次调用该接口。

步骤 4 销毁 CIPHER 句柄。调用接口 [ES_CIPHER_DestroyHandle](#) 完成。

步骤 5 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

-----结束

注意事项

- 1、对称加解密操作中每个数据包的长度不能大于 1GB。如果数据长度大于 1G，请拆分成多个数据包进行处理。
- 2、传入的地址参数为数据的用户空间虚拟地址。
- 3、加密、解密的源地址、目的地址可以相同，即可以在原地（密文、明文使用同一块 buffer）进行数据的加密和解密。

4、在进行加密、解密运算前必须先获取 CIPHER 句柄，当长时间不使用时可以释放，建议加密、解密

密各获取一个句柄，每个句柄只进行加密或者只进行解密操作。

- 5、使用非 ECB 模式进行 CIPHER 的加解密，获取控制信息的 CIPHER 句柄时，需要传入第一个包的初始化向量 IV（Initial Vector）。拆分成多个数据包加解密时，对每个包调用

[ES_CIPHER_Encrypt](#)/[ES_CIPHER_Decrypt](#)，上一个包的向量运算结果会自动作为下一个包的 IV。

- 6、不支持 CCM 和 GCM 两种模式。可通过扩展服务程序，支持这两种模式。

示例

具体示例请参见发布包 sample: sample_aes_sm4.c

1.1.2 HASH 计算

工作流程

对数据进行 HASH 计算的过程如下：

步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。

步骤 2 选择 HASH 算法，获取 HASH 句柄。调用接口 [ES_CIPHER_HashInit](#) 完成。

步骤 3 输入数据，完成 HASH 计算。调用接口 [ES_CIPHER_HashFinish](#) 完成。

步骤 4 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

-----结束

注意事项

- 1、HASH 计算输入数据最大 size 为 1GB，不支持分成多个数据块依次计算 HASH 值。
- 2、传入的地址参数为数据的用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_hash.c

1.1.3 HMAC 计算

场景说明

计算数据的 HMAC 值。HMAC 是密钥相关的哈希运算消息认证码（Hash-based Message Authentication Code）的缩写。HMAC 中的 H 指 hash 散列算法，HMAC 可以使用多种散列算法其计算结果长度与对应散列算法一致。CIPHER 硬件支持的散列算法如下：

MD5、SHA1、SHA224、SHA256、SHA384、SHA512、SHA512_224、SHA512_256、SM3、XCBC、CMAC

工作流程

对数据进行对称的 HMAC 计算的过程如下：

步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。

步骤 2 选择 HASH 算法，输入 HMAC key 和 keyLen，获取 HASH 句柄。

调用接口 [ES_CIPHER_HashInit](#) 完成。

步骤 3 输入数据，完成 HMAC 计算。调用接口 [ES_CIPHER_HashFinish](#) 完成。

步骤 4 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

----结束

注意事项

- 1、HMAC 计算输入数据最大 size 为 1GB，不支持分成多个数据块依次计算 HMAC 值。
- 2、传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_hash.c

1.1.4 产生随机数

场景说明

获取硬件产生的真随机数。

工作流程

生成随机数的过程如下：

步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。

步骤 2 传入随机数的长度（以 byte 为单位）以及随机数输出的地址，获取随机数。调用接口 [ES_CIPHER_GetRandomNumber](#) 完成。

步骤 3 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

----结束

注意事项

- 1、传入的地址参数为数据的用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_trng.c

1.1.5 RSA 加解密

场景说明

对数据进行 RSA 非对称加解密。当使用公钥加密的数据，必须使用私钥进行解密，反之，使用私钥加密的数据，必须使用公钥进行解密

工作流程

对数据进行 RSA 非对称加解密的过程如下：

步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。

步骤 2 根据使用的密钥不同，分为 4 个接口，用户可以调用以下任一接口进行加解密。

- 公钥加密 [ES_CIPHER_RsaPublicEncrypt](#)
- 私钥解密 [ES_CIPHER_RsaPrivateDecrypt](#)
- 私钥加密 [ES_CIPHER_RsaPrivateEncrypt](#)
- 公钥解密 [ES_CIPHER_RsaPublicDecrypt](#)

步骤 3 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

-----结束

注意事项

- 1、RSA 的密钥位宽可选 1024、2048、4096。加密时，采用 RSAES-PKCS1-v1_5 方式对明文做 padding。所以明文的长度必须小于 [密钥的长度 - 11]。
- 2、传入的地址参数为用户空间虚拟地址。
- 3、CIPHER 硬件内部的 ROM 上，固化了 7 个 RSA2048 公钥，不能被更改。
- 4、CIPHER 硬件内部的 OTP 上，可存储 1 个 RSA2048 公钥，只能烧写一次。
- 5、CIPHER 硬件内部的 ram 上，可存储 1 个 RSA2048 公钥，可由用户更新，更新次数不限。
- 6、CIPHER 硬件，可使用来自用户传入的公钥或者私钥（支持 1024、2048、4096 位宽），进行 RSA 加解密。

示例

具体示例请参见发布包 sample: sample_rsa.c

1.1.6 NsignVerify 签名验证

场景说明

NsignVerify 签名验证功能需要配合 PC 上的 ESWIN NSIGN 工具一同使用。首先，通过 ESWIN NSIGN 工具对 image 打包，生成包含 sign、image 的 bin 文件。再由 NsignVerify 验证 bin 文件的合法性，输出明文签名。

工作流程

对 NSIGN 工具打包后的 bin 文件，进行签名验签的过程如下：

步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。

步骤 2 从 bin 文件中获取签名和 image 的地址。调用 [ES_CIPHER_NsignFileParse](#) 接口完成。

步骤 3 选择非对称加解密算法，以及公钥在 CIPHER 硬件中存储的位置，输入签名地址和 image 地址，进行签名验证。调用 [ES_CIPHER_NsignVerify](#) 完成。

步骤 4 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

-----结束

注意事项

1、签名验证使用的公钥，只能来自 CIPHER 硬件内部的 OTP、ROM 或者 RAM 上预先存储的公钥。

如果 RAM 上预先未存储公钥，则不能选择 RAM key 作为参数，否则会导致 CIPHER 硬件工作异常。

2、CIPHER 硬件内部的公钥，只支持 RSA2048 和 ECDSA 两种非对称算法。

3、传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_sign_verify.c

1.1.7 Image 解密

场景说明

Image 解密需要配合 PC 上的 ESWIN NSIGN 工具一同使用。Image 解密的过程是在 NsignVerify 的基础上，继续对 bin 文件里面的 image 解密，得到明文数据。

工作流程

对 NSIGN 工具打包后的 bin 文件，进行 image 解密的过程如下：

步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。

步骤 2 从 bin 文件中获取签名和 image 的地址。调用 [ES_CIPHER_NsignFileParse](#) 接口完成。

步骤 3 选择非对称加解密算法，以及公钥在 CIPHER 硬件中存储的位置，输入签名地址和 image 地址，进行 image 解密。调用 [ES_CIPHER_ImageDec](#) 完成。

步骤 4 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

----结束

注意事项

1、Image 解密使用的公钥，只能来自 CIPHER 硬件内部的 OTP、ROM 或者 RAM 上预先存储的公钥。

如果 RAM 上预先未存储公钥，则不能选择 RAM key 作为参数，否则会导致 CIPHER 硬件工作异常。

2、CIPHER 硬件内部的公钥，只支持 RSA2048 和 ECDSA 两种非对称算法。

3、传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_image_dec.c

1.1.8 ECDH Key 协商

场景说明

ECDH（椭圆曲线迪菲-赫尔曼密钥交换）是一种密钥协商协议，其核心是生成双方通信的对称密钥，对称密钥生成过程中需要双向进行公钥交换，对称密钥生成后，双方基于对称密钥进行对称加密通信。

工作流程

ECDH 方式生成对称密钥的过程如下：

- 步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。
 - 步骤 2 选择椭圆曲线类别，生成非对称密钥。调用 [ES_CIPHER_EcdhKeyGen](#) 接口完成。
 - 步骤 3 根据远端的公钥，结合本地的公私钥，协商出共享的公钥。调用 [ES_CIPHER_EcdhKeyAgree](#) 接口完成。
 - 步骤 4 根据远端的随机数，结合步骤 3 协商出的共享公钥，计算对称密钥。调用 [ES_CIPHER_EcdhKeyDerivation](#) 接口完成。
 - 步骤 5 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。
- 结束

注意事项

- 1、 椭圆曲线的类型支持 NIST_P256、NIST_P384、NIST_P521、BRAINPOOL_P256R1、BRAINPOOL_P384R1、BRAINPOOL_P512R1。
- 2、 传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_ecdh.c

1.1.9 更新 RAM 公钥

场景说明

CIPHER 硬件内部 RAM 放置了一组 RSA 和一组 ECDSA 临时的公钥。其中 RSA 公钥可以用于数据的加解密或签名验签。ECDSA 公钥可以用于签名验签。这两组公钥可由用户更新，更新过程中，这两组公钥需要通过 OTP 或 ROM 上的公钥进行签名验证。首先，通过 PC 上的 NSIGN 工具，对需要更新的 RAM 公钥进行签名打包生成 bin 文件。通过该 API 接口验签后更新 CIPHER 内部的 RAM 公钥。

工作流程

更新 CIPHER 内部 RAM 公钥的过程如下：

- 步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。
- 步骤 2 从 bin 文件中获取签名和 payload 的地址。调用 [ES_CIPHER_NsignFileParse](#) 接口完成。

步骤 3 选择非对称加解密算法，以及公钥在 CIPHER 硬件中存储的位置，输入签名地址和 payload 地址，更新 RAM 上的公钥。调用 [ES_CIPHER_PubKeyDownload](#) 完成。

步骤 4 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

----结束

注意事项

- 1、验签使用的公钥，只能来自 CIPHER 硬件内部的 OTP 或者 ROM 上预先存储的公钥。
- 2、CIPHER 硬件内部的公钥，只支持 RSA2048 和 ECDSA 两种非对称算法。
- 3、传入的地址参数为用户空间虚拟地址。
- 4、CIPHER 硬件会把 payload 解密后的内容，覆盖到输入的 payload 地址。因此，请用户保存好原始 payload 的备份。

1.1.10 下载扩展服务程序

场景说明

本文所涉及的 API 都是与 CIPHER 硬件中运行的 service 服务程序通信，由 service 服务程序实际完成 API 请求的功能。用户也可以下载一段自己的 service 到 CIPHER 硬件中运行，实现扩展服务功能。例如，可通过扩展服务程序，支持 CCM 和 GCM 模式的对称加解密功能。该功能需要配合 PC 上的 ESWIN NSIGN 工具一同使用。首先，通过 ESWIN NSIGN 工具对 image 打包，生成包含 sign、image 的 bin 文件。经过验签后，image 作为 service 服务程序下载到 CIPHER 硬件运行。在下载的服务程序中，至少需要实现 4 个函数接口：

- Initialize function-用于服务注册
- Destroy function-用于服务注销
- Ioctl function-处理用户的输入
- Irqfunction-中断服务函数(可选)

工作流程

下载、调用、注销扩展服务的过程如下：

步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。

步骤 2 从 bin 文件中获取签名和 image 的地址。调用 [ES_CIPHER_NsignFileParse](#) 接口完成。

步骤 3 选择非对称加解密算法，以及公钥在 CIPHER 硬件中存储的位置，输入签名地址和 image 地址，

验签后下载 image 到 CIPHER 硬件并运行。调用 [ES_CIPHER_LoadableRegister](#) 完成。

步骤 4 调用 [ES_CIPHER_LoadableIoctl](#)，处理用户的输入。

步骤 5 调用 [ES_CIPHER_LoadableUnRegister](#)，注销服务。

步骤 6 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

----结束

注意事项

- 1、验签使用的公钥，只能来自 CIPHER 硬件内部的 OTP、ROM 或者 RAM 上预先存储的公钥。
如果 RAM 上预先未存储公钥，则不能选择 RAM key 作为参数，否则会导致 CIPHER 硬件工作异常。
- 2、CIPHER 硬件内部的公钥，只支持 RSA2048 和 ECDSA 两种非对称算法。
- 3、传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_loadable.c.

sample_loadable.c 演示签名和验签的流程图如下：

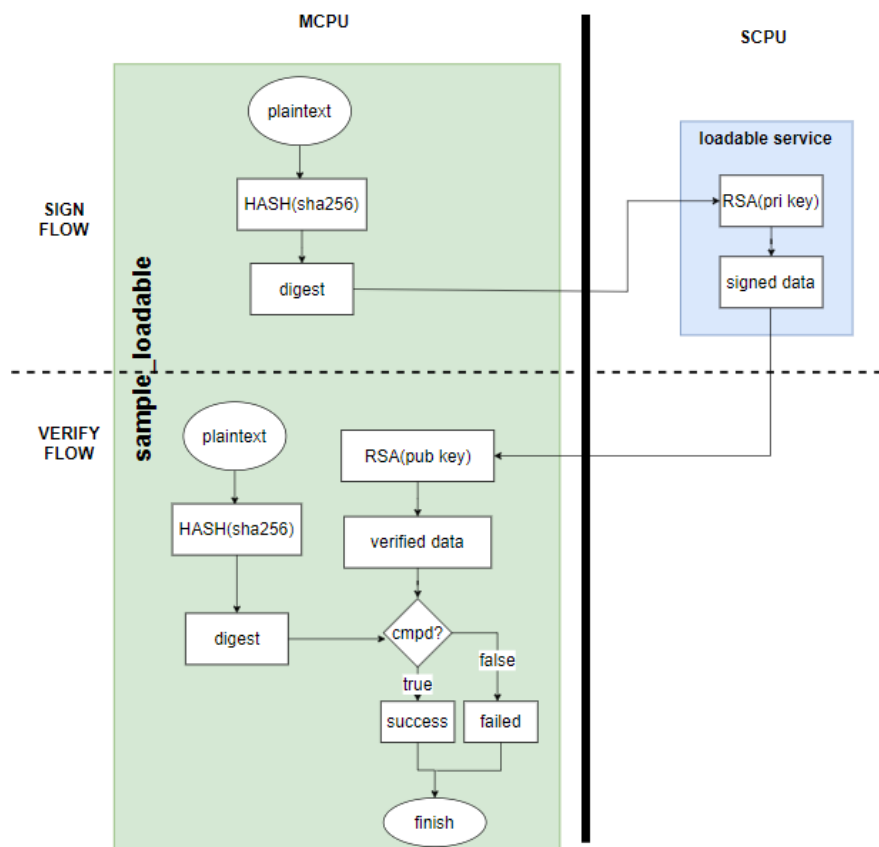


图 1- 2 扩展服务-签名/验签 流程图

其中制作 loadable 程序说明，见附一、生成 loadable 程序。

1.1.11 RSA 签名及验签操作步骤

场景说明

RSA 签名时，首先计算输入数据的 SHA256 值，再使用私钥对 SHA256 值进行加密得到签名。RSA

验签时，首先计算输入数据的 SHA256 值 A，再使用公钥对签名解密得到解密 SHA256 值 B，比较 A 和 B 的值，若相同则验签成功，否则验签失败。

工作流程

RSA 签名及验签过程如下：

步骤 1 CIPHER 设备初始化。调用接口 [ES_CIPHER_Init](#) 完成。

步骤 2 对数据进行签名或签名验证，调用以下接口完成。

- 私钥签名--[ES_CIPHER_RsaSign](#)
- 公钥验证--[ES_CIPHER_RsaVerify](#)

步骤 3 关闭 CIPHER 设备。调用接口 [ES_CIPHER_DeInit](#) 完成。

-----结束

注意事项

- 1、RSA 公钥可以来自外部或者内部。RSA 私钥只能来自外部输入。当密钥来自外部时，位宽可选 1024、2048、及 4096。当 RSA 公钥来自内部 OTP/ROM/RAM 时，位宽只能选 2048。

示例

具体示例请参见发布包 sample: sample_rsa_sign.c

2. API 参考

Cipher API 的库文件 libes_cipher.so，ibes_cipher.a，头文件：es_cipher.h

ES_CIPHER_Init

【函数体】

```
ES_S32 ES_CIPHER_Init(ES_VOID)
```

【描述】

Init the cipher device.

【参数】

None

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

ES_CIPHER_DeInit

【函数体】

```
ES_S32 ES_CIPHER_DeInit(ES_VOID)
```

【描述】

Deinit the cipher device.

【参数】

None

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

ES_CIPHER_CreateHandle

【函数体】

```
ES_S32 ES_CIPHER_CreateHandle(  
    ES\_HANDLE *phCipher,  
    ES\_CIPHER\_CTRL\_S *pstCipherAttr)
```

【描述】

Obtain a cipher handle for symmetric encryption and decryption.

【参数】

Parameter Name	描述	Input/Output
phCipher	phCipher Cipher handle	Output
pstCipherAttr	Symmetric algorithm attributes	Input

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

ES_CIPHER_DestroyHandle

【函数体】

ES_S32 ES_CIPHER_DestroyHandle(
[ES_HANDLE](#) hCipher)

【描述】

This API is used to destroy an existing cipher handle.

【参数】

Parameter Name	描述	Input/Output
hCipher	hCipher Cipher handle	Input

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

ES_CIPHER_Encrypt**【函数体】**

ES_S32 ES_CIPHER_Encrypt(
[ES_HANDLE](#) hCipher,
[ES_U8](#) *pu8SrcData,
[ES_U8](#) *pu8DestData,
[ES_U32](#) u32ByteLength)

【描述】

This API is used to perform the symmetric encryption.

【参数】

Parameter Name	描述	Input/Output
hCipher	hCipher Cipher handle	Input
pu8SrcData	buffer of the source data.	Input
pu8DestData	buffer of the target data	Output
u32ByteLength	Length of the source data	Input

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】

- The length of the source data must be aligned with 16 bytes.

ES_CIPHER_Decrypt

【函数体】

```
ES_S32 ES_CIPHER_Decrypt(  
    ES\_HANDLE hCipher,  
    ES\_U8 *pu8SrcData,  
    ES\_U8 *pu8DestData,  
    ES\_U32 u32ByteLength)
```

【描述】

This API is used to perform the symmetric decryption.

【参数】

Parameter Name	描述	Input/Output
hCipher	hCipher Cipher handle	Input
pu8SrcData	buffer of the source data.	Input
pu8DestData	buffer of the target data	Output
u32ByteLength	Length of the source data	Input

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】

- The length of the source data must be aligned with 16 bytes.

ES_CIPHER_GetHandleConfig

【函数体】

```
ES_S32 ES_CIPHER_GetHandleConfig(  
    ES\_HANDLE hCipher,  
    ES\_CIPHER\_CTRL\_S *pstCtrl)
```

【描述】

Get the cipher control information.

【参数】

Parameter Name	描述	Input/Output
hCipher	hCipher Cipher handle	Input

Parameter Name	描述	Input/Output
pstCtrl	Cipher control information.	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】

ES_CIPHER_HashInit

【函数体】

```
ES_S32 ES_CIPHER_HashInit(
    ES\_CIPHER\_HASH\_CTRL\_S *pstHashCtrl,
    ES\_HANDLE *pHashHandle)
```

【描述】

Select the HASH algorithm and return the HASH handle.

【参数】

Parameter Name	描述	Input/Output
pstHashCtrl	The hash calculating structure input	Input
pHashHandle	The output hash handle	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】

ES_CIPHER_HashFinish

【函数体】

```
ES_S32 ES_CIPHER_HashFinish(
    ES\_HANDLE hHashHandle,
    ES\_U8 *pu8InputData,
    ES\_U32 u32InputDataLen,
    ES\_U8 *pu8OutputHash,
    ES\_U32 *pu32Hlen)
```

【描述】

Calculate the HASH value.

【参数】

Parameter Name	描述	Input/Output
pstHashCtrl	Hash handle.	Input
pu8InputData	The input data buffer.	Input
u32InputDataLen	Length of the input data	Input
pu8OutputHash	The output hash result	Output
pu32Hlen	The length of the output hash result	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

- The max length of the input data is 1GB
- Multi-package, i.e. HashUpdate() is not supported.

ES_CIPHER_GetRandomNumber**【函数体】**

```
ES_S32 ES_CIPHER_GetRandomNumber(
    ES_U8 *pu8RandomNumber,
    ES_U32 u32ByteLength)
```

【描述】

This API is used to obtain the true random number from the hardware.

【参数】

Parameter Name	描述	Input/Output
pu8RandomNumber	The output of the random data	Output
pHashHandle	The lenght of the random data.	Input

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

ES_CIPHER_RsaPublicEncrypt

【函数体】

```

ES_S32 ES_CIPHER_RsaPublicEncrypt(
    ES\_CIPHER\_RSA\_PUB\_CTRL\_S *pstRsaEnc,
    ES_U8 *pu8Input,
    ES_U32 u32InLen,
    ES_U8 *pu8Output,
    ES_U32 *pu32OutLen)

```

【描述】

RSA encryption a plaintext with a RSA public key.

【参数】

Parameter Name	描述	Input/Output
pstRsaEnc	Public encryption struct	Input
pu8Input	input data to be encrypted	Input
u32InLen	length of input data to be encrypted	Input
pu8Output	output encrypted data	Output
pu32OutLen	length of output encrypted data	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

ES_CIPHER_RsaPrivateDecrypt

【函数体】

```

ES_S32 ES_CIPHER_RsaPrivateDecrypt(
    ES\_CIPHER\_RSA\_PRI\_CTRL\_S *pstRsaDec,
    ES_U8 *pu8Input,
    ES_U32 u32InLen,
    ES_U8 *pu8Output,
    ES_U32 *pu32OutLen)

```

【描述】

RSA decryption a ciphertext with a RSA private key.

【参数】

Parameter Name	描述	Input/Output
pstRsaEnc	Private decryption struct	Input
pu8Input	input data to be decrypted	Input
u32InLen	length of input data to be decrypted	Input
pu8Output	output decrypted data	Output
pu32OutLen	length of output decrypted data	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

ES_CIPHER_RsaPrivateEncrypt

【函数体】

```

ES_S32 ES_CIPHER_RsaPrivateDecrypt(
    ES\_CIPHER\_RSA\_PRI\_CTRL\_S *pstRsaEnc,
    ES\_U8 *pu8Input,
    ES\_U32 u32InLen,
    ES\_U8 *pu8Output,
    ES\_U32 *pu32OutLen)

```

【描述】

RSA encryption a plaintext with a RSA private key.

【参数】

Parameter Name	描述	Input/Output
pstRsaEnc	Private encryption struct	Input
pu8Input	input data to be encrypted	Input
u32InLen	length of input data to be encrypted	Input
pu8Output	output encrypted data	Output
pu32OutLen	length of output encrypted data	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

ES_CIPHER_RsaPublicDecrypt

【函数体】

```
ES_S32 ES_CIPHER_RsaPublicDecrypt(
    ES\_CIPHER\_RSA\_PUB\_CTRL\_S *pstRsaDec,
    ES_U8 *pu8Input,
    ES_U32 u32InLen,
    ES_U8 *pu8Output,
    ES_U32 *pu32OutLen)
```

【描述】

RSA decryption a ciphertext with a RSA public key..

【参数】

Parameter Name	描述	Input/Output
pstRsaEnc	Public decryption struct	Input
pu8Input	input data to be decrypted	Input
u32InLen	length of input data to be decrypted	Input
pu8Output	output decrypted data	Output
pu32OutLen	length of output decrypted data	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】

ES_CIPHER_NSignVerify

【函数体】

```
ES_S32 ES_CIPHER_NsignVerify(
    ES\_CIPHER\_NSIGN\_VERIFY\_S *pstNsignVerify,
    ES_U8 *pu8InData,
    ES_U32 u32InDataLen,
    ES_U8 *pu8InSign,
    ES_U32 u32InSignLen,
    ES_U8 *pu8Output,
    ES_U32 *pu32OutLen)
```

【描述】

Perform signature verification on the NSIGN bin file, output the signature structure info if success.

【参数】

Parameter Name	描述	Input/Output
pstNsignVerify	NSIGN verification struct	Input
pu8InData	input payload to be signature verification	Input
u32InDataLen	length of input data to be signature	Input
pu8InSign	encrypted signature struct info	Input
pu32InSignLen	length of encrypted signature struct info	Input
pu8Output	decrypted signature struct info	Output
pu32OutLen	length of the decrypted signature struct info	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】

ES_CIPHER_ImageDec

【函数体】

```

ES_S32 ES_CIPHER_ImageDec (
    ES\_CIPHER\_NSIGN\_VERIFY\_S *pstNsignVerify,
    ES_U8 *pu8InData,
    ES_U32 u32InDataLen,
    ES_U8 *pu8InSign,
    ES_U32 u32InSignLen,
    ES_U8 *pu8Output,
    ES_U32 *pu32OutLen)

```

【描述】

Perform signature verification on the NSIGN bin file, output the signature structure info if success.

【参数】

Parameter Name	描述	Input/Output
pstNsignVerify	NSIGN verification structure	Input
pu8InData	input payload to be sign verified	Input
u32InDataLen	length of payload	Input
pu8InSign	encrypted signature structure info	Input
pu32InSignLen	length of encrypted signature structure info	Input
pu8Output	decrypted image data	Output

Parameter Name	描述	Input/Output
pu32OutLen	length of the decrypted image data	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】

If the pu8Output==NULL, then the decrypted image will be put to the address of the pu8InData. In other words, the input payload will be overwritten by decrypted payload.

ES_CIPHER_EcdhKeyGen**【函数体】**

```
ES_S32 ES_CIPHER_EcdhKeyGen(
    ES\_CIPHER\_ECC\_CURVE\_TYPE\_E enEccCurve,
    ES\_CIPHER\_ECDH\_KEY\_S *pstEcdhKey)
```

【描述】

A pair of ECDH public and private Key Generation.

【参数】

Parameter Name	描述	Input/Output
enEccCurve	Elptic curve type	Input
pstEcdhKey	The output ECDH key which contains ECC pubkey and prikey	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】**ES_CIPHER_EcdhKeyAgree****【函数体】**

```
ES_S32 ES_CIPHER_EcdhKeyAgree(
    ES\_CIPHER\_ECC\_CURVE\_TYPE\_E enEccCurve,
```

[ES_CIPHER_ECDH_PUB_KEY_S](#) *pstRemoteEcdhPubKey,
[ES_CIPHER_ECDH_PRI_KEY_S](#) *pstLocalEcdhPriKey,
[ES_CIPHER_ECDH_PUB_KEY_S](#) *pstOutSharedEcdhPubKey)

【描述】

ECDH shared public key agreement result in a shared ECDH public key.

【参数】

Parameter Name	描述	Input/Output
enEccCurve	Elliptic curve type	Input
pstRemoteEcdhPubKey	remote ECC public key struct.	Input
pstLocalEcdhPriKey	local ECC private key struct.	Input
pstOutSharedEcdhPubKey	agreement shared ECC public key struct	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】**ES_CIPHER_EcdhKeyDerivation****【函数体】**

ES_S32 ES_CIPHER_EcdhKeyDerivation(
[ES_CIPHER_ECC_CURVE_TYPE_E](#) enEccCurve,
[ES_CIPHER_ECDH_PUB_KEY_S](#) *pstSharedEcdhPubKey,
[ES_U32](#) u32Rand,
[ES_U8](#) *pu8OutSymcKey)

【描述】

ECDH key derivation. Generate a key that can be used as a symmetric algorithm key.

【参数】

Parameter Name	描述	Input/Output
enEccCurve	Elliptic curve type	Input
pstSharedEcdhPubKey	agreement shared ECC public key struct.	Input
u32Rand	random data used in this negotiation	Input
pu8OutSymcKey	Output symmetric key	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】**ES_CIPHER_PubKeyDownload****【函数体】**

```
ES_S32 ES_CIPHER_PubKeyDownload(
    ES\_CIPHER\_NSIGN\_VERIFY\_S *pstNsignVerify,
    ES\_U8 *pu8InData,
    ES\_U32 u32InDataLen,
    ES\_U8 *pu8InSign,
    ES\_U32 u32InSignLen)
```

【描述】

update the public key in the internal RAM of CIPHER hardware.

【参数】

Parameter Name	描述	Input/Output
pstNsignVerify	NSIGN verification structure	Input
pu8InData	input payload to be signature verification	Input
u32InDataLen	length of input data to be signature	Input
pu8InSign	encrypted signature structure info	Input
pu32InSignLen	length of encrypted signature structure info	Input

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】

The content of the pu8InData will be overwritten by the decrypted data during this service. Users should keep the original input payload data well to prevent the data loss. The proper way is to malloc a buffer, copy the original payload data to this buffer, and use this buffer as the input payload.

ES_CIPHER_LoadableRegister**【函数体】**

```
ES_S32 ES_CIPHER_LoadableRegister(
```

```

ES_CIPHER_NSIGN_VERIFY_S *pstNsignVerify,
ES_U8 *pu8InData,
ES_U32 u32InDataLen,
ES_U8 *pu8InSign,
ES_U32 u32InSignLen,
ES_U32 *pu32ServiceId)

```

【描述】

loadable service code register.

【参数】

Parameter Name	描述	Input/Output
pstNsignVerify	NSIGN verification struct	Input
pu8InData	input payload to be signature verification	Input
u32InDataLen	length of input data to be signature	Input
pu8InSign	encrypted signature struct info	Input
pu32InSignLen	length of encrypted signature struct info	Input
pu32ServiceId	Returned service ID that will be used in ES_CIPHER_LoadableIoctl	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】**ES_CIPHER_LoadableUnRegister****【函数体】**

```

ES_S32 ES_CIPHER_LoadableUnRegister(
ES_CIPHER_NSIGN_VERIFY_S *pstNsignVerify,
ES_U32 u32ServiceId)

```

【描述】

loadable service code unregister.

【参数】

Parameter Name	描述	Input/Output
pstNsignVerify	NSIGN verification struct	Input
u32ServiceId	ID of the service	Input

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the _ErrCode

【注意】**ES_CIPHER_LoadableIoctl****【函数体】**

```
ES_S32 ES_CIPHER_LoadableIoctl(
    ES\_CIPHER\_NSIGN\_VERIFY\_S *pstNsignVerify,
    ES_U32 u32ServiceId,
    ES_U32 u32Cmd,
    ES_U32 u32ParamSize,
    ES_U8 *pu8Param,
    ES_U8 *pu8OutParam)
```

【描述】

loadable service ioctl call.

【参数】

Parameter Name	描述	Input/Output
pstNsignVerify	NSIGN verification struct	Input
u32ServiceId	service ID that was registered previously	Input
u32Cmd	ioctl cmd	Input
u32ParamSize	Size of the parameter	Input
pu8Param	parameter	Input
pu8OutParam	context with signed data	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the _ErrCode

【注意】**ES_CIPHER_RsaSign****【函数体】**

```
ES_S32 ES_CIPHER_RsaSign(
    ES\_CIPHER\_RSA\_PRI\_CTRL\_S *pstRsaSign,
```

```

ES_U8 *pu8InData,
ES_U32 u32InDataLen,
ES_U8 * pu8OutSign,
ES_U32 * pu32OutSignLen)

```

【描述】

RSA sign function.

【参数】

Parameter Name	描述	Input/Output
pstRsaSign	RSA private encryption ctrl info	Input
pu8InData	input context to be signature.	Input
u32InDataLen	length of input context to be signature	Input
pu8OutSign	output message of signature	Output
pu32OutSignLen	length of message of signature	Output

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

The RSA private key can only be the external key, i.e, input from user.

ES_CIPHER_RsaVerify**【函数体】**

```

ES_S32 ES_CIPHER_RsaVerify(
    ES_CIPHER_RSA_PUB_CTRL_S * pstRsaVerify,
    ES_U8 *pu8InData,
    ES_U32 u32InDataLen,
    ES_U8 * pu8InSign,
    ES_U32 u32InSignLen)

```

【描述】

RSA sign function.

【参数】

Parameter Name	描述	Input/Output
pstRsaVerify	RSA private encryption ctrl info	Input
pu8InData	input context to be signature.	Input
u32InDataLen	length of input context to be signature	Input

Parameter Name	描述	Input/Output
pu8InSign	output message of signature	Input
u32InSignLen	length of message of signature	Input

【返回值】

返回值	描述
0	Success
None 0	Fail, refers to the ErrCode

【注意】

The RSA public key could be the internal key(ROM, RAM) or externa key, i.e, input from user.

ES_CIPHER_NsignFileParse**【函数体】**

```
ES_S32 ES_CIPHER_NsignFileParse(
    ES_U8 *pu8InFileData,
    ES_U32 u32InFileDataLen,
    ES_CIPHER_NSIGN_VERIFY_S *pstOutNsignVerify,
    ES_U8 **ppu8OutSignAddr,
    ES_U32 *pu32OutSignLen,
    ES_U8 **ppu8OutPayloadAddr,
    ES_U32 *pu32OutPayloadLen)
```

【描述】

Get the address of the signature and payload from NSIGN output bin file.

【参数】

Parameter Name	描述	Input/Output
pu8InFileData	the data address of the file generated by NSIGN tool	Input
u32InFileDataLen	data length of file	Input
pstOutNsignVerify	NSIGN verification struct	Output
ppu8OutSignAddr	the signature address in the file	Output
pu32OutSignLen	length of the signature	Output
ppu8OutPayloadAddr	payload address	Output
pu32OutPayloadLen	length of the payload	Output

【返回值】

返回值	描述
-----	----

0	Success
None 0	Fail, refers to the ErrCode

【注意】

3. 数据类型

相关数据类型、数据结构定义如下：

- [ES_HANDLE](#): 定义 CIPHER 的句柄类型。
- [ES_CIPHER_ALG_MODE_E](#): 定义对称加密算法、摘要算法类型选择
- [ES_CIPHER_ENC_KEY_LENGTH_E](#): 定义对称算法的 key 长度类型选择
- [ES_CIPHER_SYMC_KEY_SRC_E](#): 定义对称算法的 key 的来源
- [ES_CIPHER_CTRL_S](#): 定义对称加解密控制信息结构体
- [ES_CIPHER_HASH_CTRL_S](#): 定义 HASH 算法控制信息结构体
- [ES_CIPHER_RSA_ENC_SCHEME_E](#): 定义非对称算法的类型选择
- [ES_CIPHER_RSA_KEY_SRC_E](#): 定义非对称算法的公钥来源以及外部公私钥选择
- [ES_CIPHER_RSA_PUB_KEY_S](#): 定义 RSA 公钥结构体
- [ES_CIPHER_RSA_PUB_CTRL_S](#): 定义 RSA 公钥加解密控制结构体
- [ES_CIPHER_RSA_PRI_KEY_S](#): 定义 RSA 私钥结构体
- [ES_CIPHER_RSA_PRI_CTRL_S](#): 定义 RSA 私钥加解密控制结构体
- [ES_CIPHER_NSIGN_VERIFY_S](#): 定义验签过程中，使用的非对称算法和 Key 来源
- [ES_CIPHER_ECC_CURVE_TYPE_E](#): 定义椭圆曲线类型选择
- [ES_CIPHER_ECDH_KEY_S](#): 定义椭圆曲线公私钥对信息
- [ES_CIPHER_ECDH_PUB_KEY_S](#): 定义椭圆曲线公钥信息
- [ES_CIPHER_ECDH_PRI_KEY_S](#): 定义椭圆曲线私钥信息

ES_HANDLE

【说明】

定义 CIPHER 的句柄类型

【定义】

```
typedef unsigned int ES_HANDLE;
```

【成员】

无

ES_CIPHER_ALG_MODE_E

【说明】

定义对称加密算法、摘要算法类型选择

【定义】

```
typedef enum esES_CIPHER_ALG_MODE_E
{
    ES_CIPHER_ALG_MODE_AES_ECB = 0x4,
    ES_CIPHER_ALG_MODE_AES_CBC,
    ES_CIPHER_ALG_MODE_AES_CTR,
    ES_CIPHER_ALG_MODE_AES_CCM,
    ES_CIPHER_ALG_MODE_AES_GCM,
    ES_CIPHER_ALG_MODE_AES_CFB,
    ES_CIPHER_ALG_MODE_AES_OFB,
    ES_CIPHER_ALG_MODE_AES_CS1,
    ES_CIPHER_ALG_MODE_AES_CS2,
    ES_CIPHER_ALG_MODE_AES_CS3,
    ES_CIPHER_ALG_MODE_3DES_CBC = 0x14,
    ES_CIPHER_ALG_MODE_3DES_ECB,
    ES_CIPHER_ALG_MODE_DES_CBC,
    ES_CIPHER_ALG_MODE_DES_ECB,
    ES_CIPHER_ALG_MODE_SM4_ECB = 0x1e,
    ES_CIPHER_ALG_MODE_SM4_CBC,
    ES_CIPHER_ALG_MODE_SM4_CFB,
    ES_CIPHER_ALG_MODE_SM4_OFB,
    ES_CIPHER_ALG_MODE_SM4_CTR,
    ES_CIPHER_ALG_MODE_SM4_CCM,
    ES_CIPHER_ALG_MODE_SM4_GCM,
    ES_CIPHER_ALG_MODE_SM4_CS1,
    ES_CIPHER_ALG_MODE_SM4_CS2,
    ES_CIPHER_ALG_MODE_SM4_CS3,

    ES_CIPHER_ALG_MODE_HASH_MD5 = 42,
    ES_CIPHER_ALG_MODE_HMAC_MD5,
    ES_CIPHER_ALG_MODE_HASH_SHA1,
    ES_CIPHER_ALG_MODE_HMAC_SHA1,
    ES_CIPHER_ALG_MODE_HASH_SHA224,
    ES_CIPHER_ALG_MODE_HMAC_SHA224,
    ES_CIPHER_ALG_MODE_HASH_SHA256,
    ES_CIPHER_ALG_MODE_HMAC_SHA256,
    ES_CIPHER_ALG_MODE_HASH_SHA384,
    ES_CIPHER_ALG_MODE_HMAC_SHA384,
    ES_CIPHER_ALG_MODE_HASH_SHA512,
    ES_CIPHER_ALG_MODE_HMAC_SHA512,
    ES_CIPHER_ALG_MODE_HASH_SHA512_224,
    ES_CIPHER_ALG_MODE_HMAC_SHA512_224,
    ES_CIPHER_ALG_MODE_HASH_SHA512_256,
    ES_CIPHER_ALG_MODE_HMAC_SHA512_256,
    ES_CIPHER_ALG_MODE_MAC_XCBC,
    ES_CIPHER_ALG_MODE_MAC_CMAC,
    ES_CIPHER_ALG_MODE_HASH_CRC32 = 66,
    ES_CIPHER_ALG_MODE_HASH_SM3 = 80,
    ES_CIPHER_ALG_MODE_HMAC_SM3,
    ES_CIPHER_ALG_MODE_MAC_SM4_XCBC,
```

```
    ES_CIPHER_ALG_MODE_MAC_SM4_CMAC,
} ES\_CIPHER\_ALG\_MODE\_E;
```

[ES_CIPHER_ENC_KEY_LENGTH_E](#)

【说明】

定义对称算法的 key 长度类型选择

【定义】

```
typedef enum esES_CIPHER_ENC_KEY_LENGTH_E
{
    ES_CIPHER_KEY_AES_128BIT = 16, /**< 128-bit key for the AES algorithm */
    ES_CIPHER_KEY_AES_192BIT = 24, /**< 192-bit key for the AES algorithm */
    ES_CIPHER_KEY_AES_256BIT = 32, /**< 256-bit key for the AES algorithm */
    ES_CIPHER_KEY_DES_3KEY = 24, /**< Three keys for the DES algorithm, TDES_KEY_192BIT */
    ES_CIPHER_KEY_SM_128BIT = 16, /**< SM1, SM4 */
    ES_CIPHER_KEY_DEFAULT = 8, /**< default key length, DES-8, SM1-48, SM4-16 */
    ES_CIPHER_KEY_LEN_MAX = 32,
    ES_CIPHER_KEY_INVALID = 0xffffffff,
} ES\_CIPHER\_ENC\_KEY\_LENGTH\_E;
```

[ES_CIPHER_SYMC_KEY_SRC_E](#)

【说明】

定义对称算法的 key 的来源

【定义】

```
typedef enum esES_CIPHER_SYMC_KEY_SRC_E
{
    ES_CIPHER_SYMC_KEY_SRC_INTERNAL = 0x0, /**< Internal ROM Key*/
    ES_CIPHER_SYMC_KEY_SRC_USER = 0xFF, /**< External User Key*/
    ES_CIPHER_SYMC_KEY_SRC_INVALID = 0xffffffff,
} ES\_CIPHER\_SYMC\_KEY\_SRC\_E;
```

[ES_CIPHER_CTRL_S](#)

【说明】

定义对称加解密控制信息结构体

【定义】

```
typedef struct esES_CIPHER_CTRL_S
{
    ES_U32 u32Key[8]; /**< Key input */
    ES_U32 u32IV[8]; /**< Initialization vector (IV) , MAX IV_LEN is 32 Bytes */
    ES\_CIPHER\_SYMC\_KEY\_SRC\_E enKeySrc; /**< Select key source: internal, or from u32Key[8] as
                                         external user key */
    ES\_CIPHER\_ALG\_MODE\_E enAlgMode; /**< Cipher algorithm */
    ES\_CIPHER\_ENC\_KEY\_LENGTH\_E enKeyLen; /**< Key length */
    ES_U32 u32IvLen;
} ES\_CIPHER\_CTRL\_S;
```

[ES_CIPHER_HASH_CTRL_S](#)

【说明】

定义 HASH 算法控制信息结构体

【定义】

```
typedef struct
{
    ES_U8 *pu8HMACKey;
    ES_U32 u32HMACKeyLen;
    ES\_CIPHER\_ALG\_MODE\_E enShaType;
} ES\_CIPHER\_HASH\_CTRL\_S;
```

[ES_CIPHER_RSA_ENC_SCHEME_E](#)

【说明】

定义非对称算法的类型选择

【定义】

```
typedef enum esES_CIPHER_RSA_ENC_SCHEME_E
{
    ES_CIPHER_RSA_SCHEME_RSA1024 = 0,
    ES_CIPHER_RSA_SCHEME_RSA2048,
    ES_CIPHER_RSA_SCHEME_RSA4096,
    ES_CIPHER_RSA_SCHEME_ECDSA,
    ES_CIPHER_RSA_SCHEME_PLAINTEXT,
} ES\_CIPHER\_RSA\_ENC\_SCHEME\_E;
```

[ES_CIPHER_RSA_KEY_SRC_E](#)

【说明】

定义非对称算法的公钥来源以及外部公私钥选择

【定义】

```
typedef enum esES_CIPHER_RSA_KEY_SRC_E
{
    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_OTP = 0x0, /**< Internal OTP Key*/

    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_ROM\_1, /**< Internal ROM Key 1*/
    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_ROM\_2, /**< Internal ROM Key 2*/
    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_ROM\_3, /**< Internal ROM Key 3*/
    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_ROM\_4, /**< Internal ROM Key 4*/
    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_ROM\_5, /**< Internal ROM Key 5*/
    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_ROM\_6, /**< Internal ROM Key 6*/
    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_ROM\_7, /**< Internal ROM Key 7*/

    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_RAM = 8, /**< Internal ram Key*/

    ES_CIPHER_RSA_PRIV_KEY_SRC_EXT = 254, /**< External private Key*/
    ES\_CIPHER\_RSA\_PUB\_KEY\_SRC\_EXT = 255, /**< External public Key*/

    ES_CIPHER_RSA_KEY_SRC_INVALID = 0xffffffff,
} ES\_CIPHER\_RSA\_KEY\_SRC\_E;
```

[ES_CIPHER_RSA_PUB_KEY_S](#)

【说明】

定义 RSA 公钥结构体

【定义】

```
typedef struct
{
    ES_U32 u32NLen;/**< length of public modulus(i.e, keylen), max value is 512Byte*/
    ES_U32 u32E; /**< public exponent */
    ES_U8 *pu8N; /**< point to public modulus */
}ES_CIPHER_RSA_PUB_KEY_S;
```

[ES_CIPHER_RSA_PUB_CTRL_S](#)

【说明】

定义 RSA 公钥加解密控制结构体

【定义】

```
typedef struct
{
    ES\_CIPHER\_RSA\_ENC\_SCHEME\_E enScheme;/** RSA encryption scheme*/
    ES\_CIPHER\_RSA\_KEY\_SRC\_E enKeySrc; /** RSA key source*/
    ES\_CIPHER\_RSA\_PUB\_KEY\_S stPubKey; /** RSA public key struct */
}ES_CIPHER_RSA_PUB_CTRL_S;
```

[ES_CIPHER_RSA_PRI_KEY_S](#)

【说明】

定义 RSA 私钥结构体

【定义】

```
typedef struct
{
    ES_U32 u32NLen;/**< length of public modulus(i.e, keylen), max value is 512Byte*/
    ES_U8 *pu8D; /*!< private exponent */
    ES_U8 *pu8N; /*!< public modulus */
}ES_CIPHER_RSA_PRI_KEY_S;
```

[ES_CIPHER_RSA_PRI_CTRL_S](#)

【说明】

定义 RSA 私钥加解密控制结构体

【定义】

```
typedef struct
{
    ES\_CIPHER\_RSA\_ENC\_SCHEME\_E enScheme;/** RSA encryption scheme */
    ES\_CIPHER\_RSA\_KEY\_SRC\_E enKeySrc; /** RSA key source*/
    ES\_CIPHER\_RSA\_PRI\_KEY\_S stPriKey; /** RSA private key struct */
}ES_CIPHER_RSA_PRI_CTRL_S;
```

[ES_CIPHER_NSIGN_VERIFY_S](#)

【说明】

定义验签过程中，使用的非对称算法和 Key 来源结构体

【定义】

```
typedef struct
{
    ES\_CIPHER\_RSA\_ENC\_SCHEME\_E enScheme;/** RSA signature scheme*/
    ES\_CIPHER\_RSA\_KEY\_SRC\_E enKeySrc; /** RSA key source*/
} ES\_CIPHER\_NSIGN\_VERIFY\_S;
```

[ES_CIPHER_ECC_CURVE_TYPE_E](#)

【说明】

定义椭圆曲线类型选择

【定义】

```
typedef enum esES_CIPHER_ECC_CURVE_TYPE_E
{
    ES_CIPHER_PKA_SW_CURVE_NIST_P256 = 0,
    ES_CIPHER_PKA_SW_CURVE_NIST_P384,
    ES_CIPHER_PKA_SW_CURVE_NIST_P521,
    ES_CIPHER_PKA_SW_CURVE_BRAINPOOL_P256R1,
    ES_CIPHER_PKA_SW_CURVE_BRAINPOOL_P384R1,
    ES_CIPHER_PKA_SW_CURVE_BRAINPOOL_P512R1,
    ES_CIPHER_PKA_SW_CURVE_INVALID
} ES\_CIPHER\_ECC\_CURVE\_TYPE\_E;
```

[ES_CIPHER_ECDH_KEY_S](#)

【说明】

定义椭圆曲线公私钥对信息

【定义】

```
typedef struct
{
    ES_U32 keylen;
    ES_U8 *pu8X;
    ES_U8 *pu8Y;
    ES_U8 *pu8Z;
} ES\_CIPHER\_ECDH\_KEY\_S;
```

[ES_CIPHER_ECDH_PUB_KEY_S](#)

【说明】

定义椭圆曲线公钥信息

【定义】

```
typedef struct
{
    ES_U32 keylen;
    ES_U8 *pu8X;
    ES_U8 *pu8Y;
} ES\_CIPHER\_ECDH\_PUB\_KEY\_S;
```

[ES_CIPHER_ECDH_PRI_KEY_S](#)

【说明】

定义椭圆曲线私钥信息

```
【定义】
typedef struct
{
    ES_U32 keylen;
    ES_U8 *pu8Z;
}ES_CIPHER_ECDH_PRI_KEY_S;
```

4. 错误码

表 4-1 错误码

ErroCode	Macro Definition	描述
0xA0136003	ES_ERR_CIPHER_INVALID_PARA	Invalid parameter
0xA0136040	ES_ERR_CIPHER_NOT_INIT	CIPHER not initialized
0xA0136041	ES_ERR_CIPHER_FAILED_INIT	Failed to initialize CIPHER
0xA0136042	ES_ERR_CIPHER_OVERFLOW	The count of CIPHER being initialized exceed the limitation
0xA0136043	ES_ERR_CIPHER_INVALID_HANDLE	The symmetric crypto handle ID is invalid
0xA0136044	ES_ERR_CIPHER_FAILED_GETHANDLE	Failed to get the handle
0xA0136045	ES_ERR_CIPHER_FAILED_RELEASEHANDLE	Failed to release the handle
0xA0136046	ES_ERR_CIPHER_HW_ERROR	CIPHER hardware err, failed to send message to CIPHER hardware
0xA0136047	ES_ERR_CIPHER_SERVICE_FAILED	Error happened for the service
0xA0136048	ES_ERR_CIPHER_IOVA	Failed to allocate or free IOVA

附一、制作 loadable 程序

下载扩展服务程序需要下载扩展服务程序到安全部件中执行，本节介绍如何制作和使用扩展服务程序。以 scpu_ldb_server/ldb_rsa_sample.c 扩展服务程序示例为介绍模板，示例完成数据 RSA2048 私钥签名，返回签名后的数据给调用程序。

扩展服务程序需要实现 4 个函数接口：Initialize function, Destroy function, ioctl function 和 Irqfunction (可选)。并将这几个接口函数地址填入到 nsign 签名工具对应的 config 文件中。

扩展服务程序生成数据，数组内容是服务程序本身的 image 数据和签名数据。需要将这个数组复制到 sample_loadable.c 对应的数组中。具体制作 loadble service 程序过程，首先进入到 scpu_ldb_server 目录下，然后执行如下命令：

- 1. 执行 make 命令，生成 scpu_ldb_rsa_sample.bin 文件。

2. 执行 make nsign_data, 生成 scpu_ldb_rsa_sample_nsign.bin, 内容包含签名数据和 image 数据。
3. 执行 make code_data, 生成 scpu_ldb_rsa_sample_nsign.h, 其中包含数组和数组的长度。

注意：目前 sample 代码中，nsign 对 loadable server 签名，其中 image 是 plaintext。

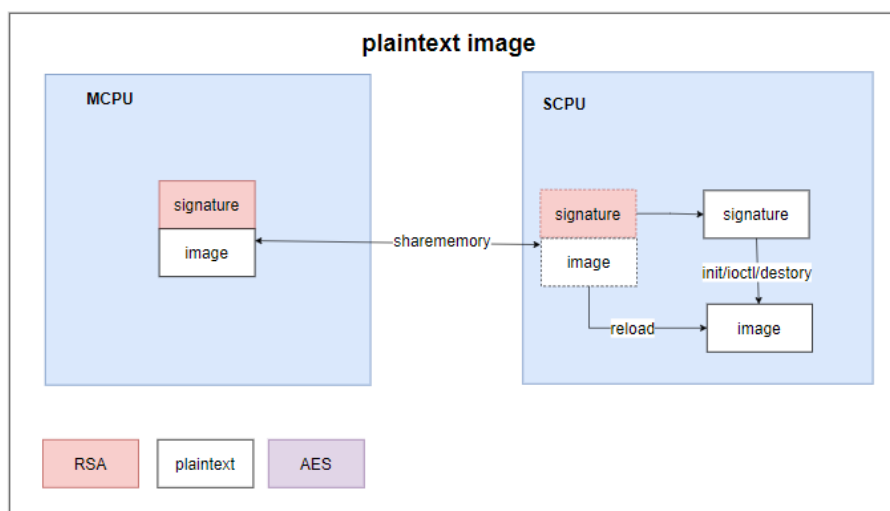


图 1- 3 nsign 签名后的数据运行演示